

23CSE111
OBJECT ORIENTED PROGRAMMING
LAB SESSION
10 JAVA PROBLEMS

QUESTION-1

AIM: To Write a Java program that demonstrates the use of classes and objects. This program defines a class called Person with attributes like name, age, and a method displayInfo() to display the information of a person

ALGORITHM:

1. Define a class person with instance variables name (String) and age (int).
2. Create a constructor person(String name, int age) to initialize the instance variables using the this keyword.
3. Define a method displayinfo() to print the name and age of the person.
4. In the main class, create objects person1 and p2 using the constructor with sample values.
5. Call the displayinfo() method for both objects to display their details.
6. Execute the program to verify that the output correctly displays the name and age.

CODE:

```
class person{  
    String name;  
    int age;  
    person(String name,int age) {
```

```
        this.name=name;
        this.age=age;
    }
    void displayinfo(){
        System.out.println("name:"+name);
        System.out.println("age:"+age);
    }
}
public class main{
    public static void main(String[] args){
        person person1=new person("alice",24);
        person p2=new person("aryan",30);
        person1.displayinfo();
        p2.displayinfo();
    }
}
```

OUTPUT:

```
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
name:alice
age:24
name:aryan
age:30
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> 
```

QUESTION-2

AIM: To write a java program to create a BankAccount class with attributes like accountNumber, balance, and methods like deposit() and withdraw(). Create objects for different bank accounts and perform deposit and withdrawal operations

ALGORITHM:

1. Define a class bankaccount with private attributes accountnum (int) and balance (double).
2. Create a constructor bankaccount(int accountnum, double balance) to initialize the account number and balance.
3. Define a deposit(double amount) method that adds the amount to the balance if it is valid.
4. Define a withdraw(double amount) method that subtracts the amount from the balance if sufficient funds are available.
5. Define a display() method to print the account number and current balance.
6. In the main class, create two bankaccount objects (b1 and b2) with initial balances.
7. Perform transactions by calling deposit(), withdraw(), and display() methods for both accounts.
8. Print transaction details for both accounts.
9. Execute the program to verify correct account operations and balance updates.

CODE:

```
class bankaccount{  
    private int accountnum;  
    private double balance;  
    bankaccount(int accountnum,double balance){
```

```

        this.accountnum=accountnum;
        this.balance=balance;
    }
    public void deposit(double amount){
        if (amount>0){
            balance += amount;
            System.out.println("deposited="+balance);}
        else{
            System.out.println("invalid deposit amount");}
    }
    public void withdraw(double amount){
        if (amount>0 && amount<=balance){
            balance -= amount;
            System.out.println("withdrawn="+balance);}
        else{
            System.out.println("invalid withdraw amt");}
    }
    public void display(){
        System.out.println("Account number="+accountnum);
        System.out.println("current balance="+balance);
    }
}

public class main{
    public static void main(String[] args){
        bankaccount b1=new bankaccount(101,1000);
        bankaccount b2=new bankaccount(202,5000);

        System.out.println("Transaction in account-1");
        b1.deposit(1000);
        b1.withdraw(100);
        b1.display();
        System.out.println();
        System.out.println("Transaction in bank account 2");
        b2.deposit(100);
        b2.withdraw(10000);
        b2.display();
    }
}

```

OUTPUT:

```
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
Transaction in account-1
deposited=2000.0
withdrawn=1900.0
Account number=101
current balance=1900.0

Transaction in bank account 2
deposited=5100.0
invalid withdraw amt
Account number=202
current balance=5100.0
```

QUESTION-3

AIM:

Hierarchical Inheritance: Create a base class Shape with methods to calculate area and perimeter. Create derived classes Rectangle and Circle that inherit from Shape.

ALGORITHM:

1. Define an abstract class shape with two abstract methods:
calculatearea() and calculateperimeter().
2. Create a subclass rectangle that extends shape and has private attributes length and width.
 - Implement the constructor to initialize length and width.
 - Override calculatearea() to return length * width.
 - Override calculateperimeter() to return 2 * (length + width).
3. Create a subclass circle that extends shape and has a private attribute radius and a constant pi = 3.14.
 - Implement the constructor to initialize radius.
 - Override calculatearea() to return pi * radius * radius.
 - Override calculateperimeter() to return 2 * pi * radius.
4. In the main class, create objects of rectangle and circle with sample values.
5. Call the calculatearea() and calculateperimeter() methods for both objects.
6. Print the calculated area and perimeter for both rectangle and circle.
7. Execute the program to verify correct calculations and output.

CODE:

```
abstract class shape{
    abstract double calculatearea();
    abstract double calculateperimeter();
}

class rectangle extends shape{
    private double length,width;
    rectangle(double length,double width){
        this.length=length;
        this.width=width;
    }
    @Override
    double calculatearea(){
        return length*width;
    }
    @Override
    double calculateperimeter(){
        return 2*(length+width);
    }
}

class circle extends shape{
    private double radius;
    private static final double pi=3.14;

    circle(double radius){
        this.radius=radius;
    }
    @Override
    double calculatearea(){
        return pi*radius*radius;
    }

    @Override
    double calculateperimeter(){
        return 2*pi*radius;
    }
}
```

```
public class main{  
    public static void main(String[] args){  
        rectangle rect=new rectangle(10,5);  
        circle c=new circle(5);  
        System.out.println("rectangle area:"+rect.calculatearea());  
        System.out.println("rectangle  
perimeter:"+rect.calculateperimeter());  
  
        System.out.println("circle area:"+rect.calculatearea());  
        System.out.println("circle perimeter:"+rect.calculateperimeter());  
    }  
}
```

OUTPUT:

```
rectangle area:50.0  
rectangle perimeter:30.0  
rectangle area:50.0  
rectangle perimeter:30.0
```


QUESTION-4

AIM:To write any program using Multilevel Inheritance

ALGORITHM:

1. Define the `Vehicle` class with an attribute `brand` and a constructor to initialize it.
2. Create a method `displayInfo()` in `Vehicle` to print the brand name.
3. Define a subclass `Car` that extends `Vehicle` and adds an attribute `speed`.
 - Use a constructor to initialize `brand` (via `super()`) and `speed`.
 - Implement `displayCarInfo()` to call `displayInfo()` and print the speed.
4. Define another subclass `ElectricCar` that extends `Car` and adds `batteryCapacity`.
 - Use a constructor to initialize `brand`, `speed`, and `batteryCapacity`.
 - Implement `displayElectricCarInfo()` to call `displayCarInfo()` and print battery capacity.
5. In the main method, create an object of `ElectricCar` (`tesla`) and initialize it with values.
6. Call `displayElectricCarInfo()` on the object to display all details (`brand`, `speed`, `battery capacity`).
7. Execute the program to print the information of the electric car.

CODE:

```
class Vehicle {
    String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    void displayInfo() {
        System.out.println("Brand: " + brand);
    }
}

class Car extends Vehicle {
    int speed;

    Car(String brand, int speed) {
        super(brand);
        this.speed = speed;
    }

    void displayCarInfo() {
        displayInfo();
        System.out.println("Max Speed: " + speed + " km/h");
    }
}

class ElectricCar extends Car {
    int batteryCapacity;

    ElectricCar(String brand, int speed, int batteryCapacity) {
        super(brand, speed);
        this.batteryCapacity = batteryCapacity;
    }

    void displayElectricCarInfo() {
        displayCarInfo();
        System.out.println("Battery Capacity: " + batteryCapacity + "
kWh");
    }
}
```

```
}  
  
public class main {  
    public static void main(String[] args) {  
        ElectricCar tesla = new ElectricCar("Tesla", 250, 100);  
        tesla.displayElectricCarInfo();  
    }  
}
```

OUTPUT:

```
C:\Users\thinkbook\OneDrive>  
Brand: Tesla  
Max Speed: 250 km/h  
Battery Capacity: 100 kWh
```

QUESTION-5

AIM: Write a java program using polymorphism to display the location of the hillstation and the place is famous for.

Location is: Manali is in Himachal Pradesh

Famous for: It is Famous for Hadimba Temple and adventure sportswe are creating one superclass Hillstations and three subclasses Manali, Mussoorie, Gulmarg. Subclasses extend the superclass and override its location() and famousfor() method. We will call the location() and famousfor() method by the Parent class', i.e. Hillstations class. As it

refers to the base class object and the base class method overrides the superclass method; the base class method is invoked at runtime.

ALGORITHM:

1. Define a base class hillstations with methods location() and famousFor() that print default messages.
2. Create a subclass Manali that extends hillstations and overrides location() and famousFor() to display specific details about Manali.
3. Create a subclass Mussoorie that extends hillstations and overrides location() and famousFor() to display specific details about Mussoorie.
4. Create a subclass Gulmarg that extends hillstations and overrides location() and famousFor() to display specific details about Gulmarg.
5. Define the main class with a main() method to execute the program.

6. Create objects of Manali, Mussoorie, and Gulmarg using hillstations references (hillstations hill1 = new Manali(); etc.).
7. Print hill station details and call the location() and famousFor() methods for each object.
8. Execute the program, demonstrating polymorphism, as overridden methods are called dynamically based on the actual object type.

CODE:

```
class hillstations{
    void location(){
        System.out.println("location information not available");
    }
    void famousFor(){
        System.out.println("Famous attractions are not specified.");
    }
}
class Manali extends hillstations{
    @Override
    void location(){
        System.out.println("Manali is in Himachal Pradesh.");
    }
    @Override
    void famousFor(){
        System.out.println("It is famous for Hadimba Temple and adventure sports.");
    }
}
class Mussoorie extends Hillstations {
    @Override
    void location() {
        System.out.println("Mussoorie is in Uttarakhand.");
    }
}
```

```

        @Override
        void famousFor() {
            System.out.println("It is famous for its scenic beauty and the
Mall Road.");
        }
    }
}
class Gulmarg extends Hillstations {
    @Override
    void location() {
        System.out.println("Gulmarg is in Jammu & Kashmir.");
    }

    @Override
    void famousFor() {
        System.out.println("It is famous for skiing and the Gulmarg
Gondola.");
    }
}
public class main{
    public static void main(String[] args){
        hillstations hill1=new Manali();
        hillstations hill2=new Mussoorie();
        hillstations hill3=new Gulmarg();

        System.out.println("Details of hillstations:");

        System.out.println("Manali:");
        hill1.location();
        hill1.famousFor();

        System.out.println("Mussoorie:");
        hill2.location();
        hill2.famousFor();

        System.out.println("Gulmarg:");
        hill3.location();
        hill3.famousFor();
    }
}

```

OUTPUT:

```
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
Hill Station Details:

**Manali**
Location: Manali is in Himachal Pradesh.
Famous for: Hadimba Temple and adventure sports.
```

QUESTION-6

AIM: A boy has his money deposited \$1000, \$1500 and \$2000 in banks-Bank A, Bank B and Bank C respectively. We have to print the money deposited by him in a particular bank.

Create a class 'Bank' with a method 'getBalance' which returns 0. Make its three subclasses named 'BankA', 'BankB' and 'BankC' with a method with the same name 'getBalance' which returns the amount deposited in that particular bank. Call the method 'getBalance' by the object of each of the three banks.

ALGORITHM:

1. Define a base class Bank with a method `getBalance()` that returns 0.
2. Create a subclass BankA that extends Bank and overrides `getBalance()` to return 1000.
3. Create a subclass BankB that extends Bank and overrides `getBalance()` to return 1500.
4. Create a subclass BankC that extends Bank and overrides `getBalance()` to return 2000.
5. Define the main class with a `main()` method to execute the program.
6. Create objects of BankA, BankB, and BankC using Bank references (`Bank bankA = new BankA();` etc.).
7. Call `getBalance()` on each object and print the respective balances.
8. Execute the program, demonstrating polymorphism, as the method `getBalance()` is called dynamically based on the object's actual type.

CODE:

```
class Bank {
    int getBalance() {
        return 0;
    }
}

class BankA extends Bank {
    @Override
    int getBalance() {
        return 1000;
    }
}

class BankB extends Bank {
    @Override
    int getBalance() {
        return 1500;
    }
}

class BankC extends Bank {
    @Override
    int getBalance() {
        return 2000;
    }
}

public class main {
    public static void main(String[] args) {
        Bank bankA = new BankA();
        Bank bankB = new BankB();
        Bank bankC = new BankC();

        System.out.println("Money in Bank A: $" + bankA.getBalance());
        System.out.println("Money in Bank B: $" + bankB.getBalance());
        System.out.println("Money in Bank C: $" + bankC.getBalance());
    }
}
```

OUTPUT:

```
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
Money in Bank A: $1000
Money in Bank B: $1500
Money in Bank C: $2000
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> |
```

QUESTION-7

AIM:

Single Inheritance: Create a base class called Vehicle with attributes like make, model, and methods like start(), stop(). Create a derived class Car that inherits from Vehicle and adds a method drive().

ALGORITHM:

1. Define a base class vehicle with instance variables s (model) and m (make).
2. Implement a method make(String m) to assign and print the vehicle make.
3. Implement a method model(String s) to assign and print the vehicle model.
4. Implement a method start() to print "start".
5. Implement a method stop() to print "stop".
6. Define a subclass car that extends vehicle and includes a drive() method that prints "drive".
7. In the main class, create a car object (c) and call make(), model(), start(), stop(), and drive().
8. Execute the program, demonstrating inheritance by using both vehicle and car class methods.

CODE:

```
class vehicle{
    String s;
    String m;
    void make(String m) {
        this.m=m;
        System.out.println("make:"+m);
    }
    void model(String s) {
```

```

        this.s=s;
        System.out.println("model:"+s);
    }
    void start(){
        System.out.println("start");
    }
    void stop(){
        System.out.println("stop");
    }
}
class car extends vehicle{
    void drive(){
        System.out.println("drive");
    }
}
public class main{
    public static void main(String[] args){
        car c=new car();
        c.make("abc");
        c.model("xyz");
        c.start();
        c.stop();
        c.drive();
    }
}

```

OUTPUT:

```

PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
make:abc
model:xyz
start
stop
drive

```

QUESTION-8

AIM:Create a base class Animal with a method `makeSound()`.

Create derived classes Dog and Cat that override `makeSound()` to print their respective sounds.

In the main method, create an array of Animal objects and call `makeSound()` for each.

ALGORITHM:

1. Create a base class Animal with a method `makeSound()` that prints a generic sound message.
2. Create a subclass Dog that extends Animal and overrides `makeSound()` to print "Woof Woof!".
3. Create a subclass Cat that extends Animal and overrides `makeSound()` to print "Meow Meow!".
4. Define the main class with a `main()` method to execute the program.
5. Create an array of Animal references storing objects of Dog, Cat, and Animal.
6. Use a for-each loop to iterate through the array and call `makeSound()` dynamically (polymorphism).

CODE:

```
class Animal {  
    void makeSound() {  
        System.out.println("Some generic animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void makeSound() {
```

```

        System.out.println("Dog: Woof Woof!");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Cat: Meow Meow!");
    }
}

public class main {
    public static void main(String[] args) {
        Animal[] animals = {new Dog(), new Cat(), new Animal()};
        for (Animal a : animals) {
            a.makeSound();
        }
    }
}

```

OUTPUT:

```

PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
Dog: Woof Woof!
Cat: Meow Meow!
Some generic animal sound
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> 

```

QUESTION-9

AIM: Create a base class **Vehicle** with a method `fuelType()`.

Create derived classes **Car** and **Bike**, each **overriding** `fuelType()` to specify whether they run on petrol, diesel, or electric.

Demonstrate polymorphism using an **array of Vehicle objects**.

ALGORITHM:

1. **Create a base class Vehicle** with a method `fuelType()` that prints a generic fuel message.
2. **Create a subclass Car** that extends `Vehicle` and overrides `fuelType()` to specify petrol or diesel.
3. **Create a subclass Bike** that extends `Vehicle` and overrides `fuelType()` to specify petrol.
4. **Define the main class** with a `main()` method to execute the program.
5. **Create an array of Vehicle references** storing objects of `Car`, `Bike`, and `Vehicle`.
6. **Use a for-each loop** to iterate through the array and call `fuelType()` dynamically (polymorphism).

CODE:

```
class Vehicle {
    void fuelType() {
        System.out.println("Vehicle: Generic Fuel Type");
    }
}

class Car extends Vehicle {
    @Override
    void fuelType() {
        System.out.println("Car: Runs on Petrol or Diesel");
    }
}
```

```

    }
}

class Bike extends Vehicle {
    @Override
    void fuelType() {
        System.out.println("Bike: Runs on Petrol");
    }
}

public class main {
    public static void main(String[] args) {
        Vehicle[] vehicles = {new Car(), new Bike(), new Vehicle()};

        for (Vehicle v : vehicles) {
            v.fuelType();
        }
    }
}

```

OUTPUT:

```

PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
Car: Runs on Petrol or Diesel
Bike: Runs on Petrol
Vehicle: Generic Fuel Type

```


QUESTION-10

AIM:Create a base class Employee with a method `calculateBonus()`. Create derived classes Developer and Manager, each overriding `calculateBonus()` to provide different bonus amounts.Create an array of Employee references and call `calculateBonus()` for each.

ALGORITHM:

1. Define the Employee class with a salary attribute and a constructor to initialize it.
2. Implement the `calculateBonus()` method in Employee, which calculates a 10% bonus.
3. Create a subclass Developer that extends Employee and overrides `calculateBonus()` to calculate a 20% bonus.
4. Create another subclass Manager that extends Employee and overrides `calculateBonus()` to calculate a 30% bonus.
5. In the main method, create an array of Employee objects consisting of a Developer, Manager, and a generic Employee.
6. Use a loop to iterate through the array and call `calculateBonus()` for each object.
7. Execute the program to print the respective bonuses based on the type of employee.

CODE:

```
class Employee {
    double salary;

    Employee(double salary) {
        this.salary = salary;
    }

    void calculateBonus() {
        System.out.println("Employee Bonus: " + (0.1 * salary));
    }
}
```

```

}

class Developer extends Employee {

    Developer(double salary) {
        super(salary);
    }

    @Override
    void calculateBonus() {
        System.out.println("Developer Bonus: " + (0.2 * salary));
    }
}

class Manager extends Employee {

    Manager(double salary) {
        super(salary);
    }

    @Override
    void calculateBonus() {
        System.out.println("Manager Bonus: " + (0.3 * salary));
    }
}

// Main class
public class main {
    public static void main(String[] args) {
        Employee[] employees = {new Developer(50000), new Manager(80000),
new Employee(40000)};
        for (Employee e : employees) {
            e.calculateBonus();
        }
    }
}

```

OUTPUT:

```
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> javac main.java
PS C:\Users\ThinkBook\OneDrive\Desktop\java program> java main
Developer Bonus: 10000.0
Manager Bonus: 24000.0
Employee Bonus: 4000.0
```